Master of Science in Informatics at Grenoble
Master Informatique
Specialization MoSIG M2

Efficient Hybrid Query Processing with Structured Pruning and LLM Cascades

Ann Remizova

Academic year 2025–2026

Research project performed at LIG

Under the supervision of: TODO

Defended before a jury composed of: TODO2026

Abstract

This report studies natural-language queries over heterogeneous sources containing relational movie metadata and free-text reviews. The objective is to retain answer quality while reducing the latency and number of large-language-model calls. I reimplemented two baselines—a SUQL structured-first executor and a Trummer-style adaptive semantic block join—and developed two cheap-to-expensive cascades. The most effective system first evaluates structured predicates, joins movies and reviews by exact identifiers, classifies candidate pairs in cheap-model batches, and coalesces uncertain candidates into larger strong-model batches. I formalize the routing and online threshold selection, document the prompts and failure semantics, and build an auditable IMDb benchmark. In a preliminary five-query run, the batched cascade reaches macro F1 0.701 at 21.84 seconds and 10 model calls per question. This is the best quality and efficiency point among the four tested systems, but the single run and lexical ground truth limit the conclusion.

Résumé

Ce rapport traite des requêtes en langue naturelle sur des données hétérogènes : métadonnées relationnelles de films et critiques textuelles. Deux baselines (SUQL et jointure sémantique adaptative) et deux cascades de modèles sont implémentées. La meilleure approche filtre d'abord les attributs structurés, effectue une jointure exacte par identifiant, puis combine des lots traités par un petit modèle et un repli vers un modèle plus puissant. Sur cinq requêtes, elle obtient un F1 macro de 0,701 en 21,84 secondes et 10 appels par requête. Ces résultats restent préliminaires car l'expérience ne comporte qu'une répétition et la vérité terrain est lexicale.

Contents

— 1 —
Introduction

Modern data applications rarely store all useful information in one representation. A movie catalogue contains exact attributes such as year, runtime, director, and genre, while its reviews contain opinions that cannot be recovered reliably with SQL string matching. The motivating query therefore combines both: "Which movies released in 2001 have reviews that explicitly recommend the movie?"

The project objective was to produce machine-readable answer rows from such questions with three simultaneous goals: high precision and recall, low wall-clock latency, and few expensive model calls. The initial systems exposed a trade-off. SUQL produced relatively good answers by executing structured predicates before row-wise semantic calls, but was slow. A semantic block join placed many rows in a prompt and supported separate sources, but its answer quality and unnecessary work on structured mismatches were problematic.

The work progressed from reproducing these baselines to adding two-level model cascades. SUQL V1 retains row-wise execution and routes uncertain log-odds scores to a strong model. Trummer V1 composes structured pruning, an exact key join, cheap batches, and coalesced strong-model batches. I also developed a canonical benchmark, ground-truth annotations, common runners, model-call instrumentation, and plots.

The main research question is: *Can relational pruning and batched heterogeneous model routing jointly improve the quality–latency–call trade-off of hybrid semantic queries?* Subsidiary questions concern how confidence should be defined, how a threshold can be selected without a labelled calibration set, and which failures must escalate or fail closed.

The report first reviews related execution models, then formalizes the design, documents the implementation and prompts, evaluates four systems, and closes with limitations and a roadmap. All numerical claims identify the exact experiment from which they originate.

— 2 —
State of the Art

2.1 Hybrid semantic query languages

SUQL extends SQL with ANSWER and SUMMARY functions over free-text columns and translates conversational input using in-context learning [4]. Its representation is expressive, but a physical executor can still invoke an NLP model once per tuple. BlendSQL similarly embeds external reasoning in relational algebra [3]. This project operates at the physical-plan level: it does not propose a new language.

2.2 Semantic joins and batch prompting

Trummer's semantic block join is inspired by block nested-loop join and includes multiple rows from both inputs in one prompt [5]. Batch prompting amortizes fixed instructions and request overhead and can reduce time and token cost, but large batches risk omitted identifiers and interference [2]. Our join key is not semantic: movie and review identifiers must be exactly equal. Only the residual review predicate belongs in the LLM.

2.3 Model cascades

Model cascades let a cheap model answer easy cases and escalate difficult cases. FrugalGPT demonstrates that learned cascades can reduce inference cost while maintaining quality [1]. In this project, a large model labels an online calibration sample because no per-predicate labelled training set is available at execution time. This estimates agreement with the large model, not ground-truth correctness; the distinction is central to the limitations.

2.4 Positioning

The proposed plan composes three orthogonal ideas. Relational selection reduces the candidate cardinality, batching reduces requests per candidate, and cascading reduces the fraction handled by the large model. The comparison includes complete end-to-end systems, so a future factorial ablation is needed to attribute gains causally to individual components.

— 3 —

Formal Model and Architecture

3.1 Query semantics

Let M be structured movie rows and R review rows. Their keys are k_M and k_R. A question is decomposed into a deterministic predicate p_s and a semantic predicate p_u. The answer is

$$A = \{k_M(m) \mid m \in M,\ p_s(m),\ \exists r \in R : k_M(m) = k_R(r) \wedge p_u(r)\}.$$

Precision, recall and F1 are computed over unique movie identifiers, not joined review rows. Deduplication is therefore part of the output semantics.

3.2 Architecture

The parser is conservative. It removes semantic `answer()` clauses and executes the remaining SQL in SQLite. If model parsing fails, regular expressions extract supported year, runtime, genre, director, and title constraints. Applying an unsafe predicate would cause irrecoverable false negatives, so unsupported constraints are left to the semantic stage.

3.3 Cascade mathematics

For SUQL V1 the cheap-model score is ideally

$$s_i = \log \frac{P_c(\text{YES} \mid x_i, q)}{P_c(\text{NO} \mid x_i, q)}, \quad \hat{y}_i = \mathbf{1}[s_i \geq 0], \quad c_i = |s_i|.$$

The implementation extracts both token log-probabilities where available. When only one emitted label exists, it derives a signed logit from that probability; without probability metadata it uses magnitude 2.

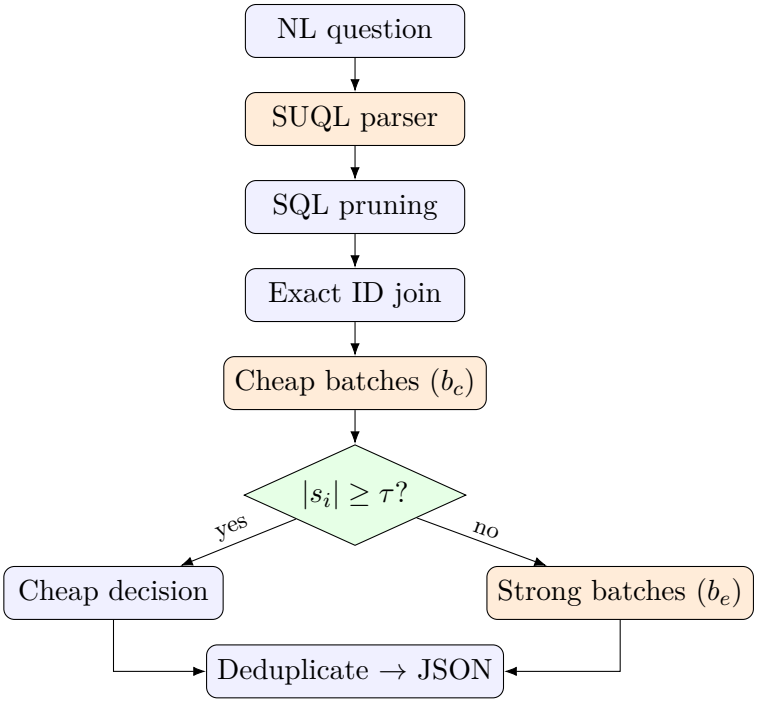

Figure 3.1: Current Trummer V1 physical architecture.

Trummer V1 returns categories and maps YES, UNCERTAIN, NO to $+2, 0, -2$. Therefore $c_i \in \{0, 2\}$. This is not a probability and currently permits only a coarse gate. A continuous per-pair score is an important future improvement.

Let S contain at most B candidates sampled approximately uniformly over the ranked confidence range. The strong model supplies labels y_i^e. At threshold t, empirical conditional agreement is

$$\widehat{a}(t) = \frac{\sum_{i \in S} \mathbf{1}[c_i \geq t]\mathbf{1}[\hat{y}_i = y_i^e]}{\sum_{i \in S} \mathbf{1}[c_i \geq t]}.$$

The algorithm scans observed t values from low to high and returns the first with $\widehat{a}(t) \geq \alpha$ (default $\alpha = .9$). If no value qualifies, $\tau = \perp$ and no cheap decision is trusted. This is an empirical agreement constraint with no finite-sample confidence bound.

For n exact candidate pairs, cheap batch size b_c, expensive batch size b_e, calibration size m, and $u(\tau)$ fallback candidates, Trummer V1 uses

$$C = \lceil n/b_c \rceil + \lceil m/b_e \rceil + \lceil u(\tau)/b_e \rceil.$$

SUQL V1 is row-wise: its corresponding terms are $n+m+u(\tau)$ calls. This explains why fewer large-model calls do not necessarily imply lower latency.

3.4 Pseudocode

```
def structured_batched_cascade(question, movies, reviews):
    ps, pu = parse_question_conservatively(question)
    movies = relational_select(movies, ps)
    X = exact_key_join(movies, reviews)
    scores = {}
    for batch in blocks(X, cheap_batch_size):
        try: scores.update(cheap_model(batch, pu))
        except InvalidResponse: mark_missing(batch)
    S = confidence_stratified_sample(scores, budget)
    oracle = strong_model_in_batches(S, pu)
    tau = least_threshold_with_agreement(scores, oracle, 0.9)
    accepted, uncertain = route(scores, missing, tau)
    for batch in blocks(uncertain, expensive_batch_size):
        accepted += strong_model(batch, pu).positive_rows
    return deduplicate_by_movie_id(accepted)
```

— 4 —

Implementation

4.1 Four execution engines

The shared runner exposes four flags. `suql_baseline` executes a supplied SUQL query, runs the structured SQL first, and calls the expensive model per surviving review. `suql_v1` replaces that answer function with the row-wise log-odds cascade. `trummer_baseline` sends blocks from both complete sources to one expensive model and adapts block sizes. `trummer_v1` applies structured pruning, exact key restriction, and the batched cascade.

The benchmark runner resets caches between repetitions, sets model and threshold configuration through environment variables, writes per-run metrics, and evaluates unique returned IDs against ground truth. Calls are separated into cheap, calibration-expensive, and fallback-expensive counts; wall and model times are recorded separately.

4.2 Routing and failure handling

SUQL caches by the MD5 hash of review and question. Empty reviews are rejected without a call. A cheap scoring exception escalates the row. Expensive requests retry transport errors; malformed output is reprompted once and then becomes NO.

Trummer assigns stable candidate IDs after exact joining. A cheap response must cover every ID; otherwise the entire affected batch is treated as missing and escalated. Expensive responses must likewise cover every ID. An exception yields no accepted IDs for that batch. These fail-closed policies favor precision and availability but can create false negatives, and are visible in the metrics.

4.3 Data and reproducibility

The repository provides 1-, 3-, 5-, and 10-question suites. Each question directory contains structured movies, reviews, their joined form, annotations, ground truth, and a JSON specification containing the natural-language question, SUQL query, semantic predicate, and expected IDs. A single registry and evaluator prevent method-specific scoring code. Local and cluster launchers select cheap/expensive models and repetitions.

4.4 Engineering lessons

Three details strongly affected behavior. First, prompts must bind every decision to an input ID; free prose is difficult to parse and can hallucinate rows. Second, calibration calls are real costs and must not be hidden. Third, model-call count is insufficient: SUQL V1's one-token cheap requests were unexpectedly slow in the reported setup, so latency and tokens must be measured directly.

— 5 —

Experimental Evaluation

5.1 Workload and protocol

The main snapshot contains five IMDb-derived queries: recommendation in 2001; fear in Horror films under 100 minutes; visual praise in Fantasy; slow/boring pacing for directors containing "John"; and originality for titles containing "The". Each question has 100 movies, 40 structured matches, and 12 ground-truth answers. Ground truth is produced by deterministic lexical patterns and audited with evidence excerpts and rationales. The compared models are Gemma 4 e2b and Gemma 4 26b at temperature zero. Default batch sizes are 8 and 32, calibration budget 20, and agreement target .9.

The presented snapshot has one repetition. Quality values are macro averages over the five questions; time and calls are means per question. This is useful for implementation validation, not statistical significance.

Table 5.1: Five-query preliminary result.

Method	Precision	Recall	F1	Seconds	Calls
SUQL baseline	.537	.567	.520	93.39	137.8
SUQL V1	.617	.717	.638	419.12	68.0
Trummer baseline	.606	.333	.396	40.41	100.4
Trummer V1	**.786**	.667	**.701**	**21.84**	**10.0**

5.2 Interpretation

Against SUQL baseline, Trummer V1 is 4.28 times faster, uses 13.78 times fewer calls, improves precision by 46.4% relative, recall by 17.6%, and F1 by 34.8%. Against Trummer baseline, its F1 improves by 77.0%, time falls by 46.0%, and calls by 90.0%. These are ratios of the displayed macro/mean values.

SUQL V1 demonstrates the value and cost of routing separately: its F1 rises 22.7% relative to SUQL, but latency increases 4.49 times. It performs 40 row-wise cheap calls and 28 strong calls on average. Trummer V1 performs five cheap batches and five strong batches. The result supports the combined plan, not the proposition that every cascade is faster.

Per-query Trummer V1 F1 ranges from .25 to .88. The originality query remains a clear weakness, while the visual and pacing predicates perform well. Aggregate scores therefore conceal predicate sensitivity.

5.3 Threats to validity

The single repetition provides no variance estimate. The ground truth favors lexical evidence and may not represent human semantic judgments. All data comes from IMDb, model versions and serving hardware affect latency, and mean call count does not measure prompt tokens, energy, or monetary cost. The large model used for routing calibration is not an oracle. Finally, the methods differ in more than one component, so causal attribution requires ablations.

5.4 Required confirmatory study

A publication-grade study should run at least ten repetitions, report per-question distributions and paired bootstrap confidence intervals, freeze model digests and software commit, record input/output tokens and hardware utilization, and use independent dual annotation with adjudication. Ablations should independently toggle structured pruning, exact joining, cheap batching, strong coalescing, and online calibration. Threshold sweeps should report Pareto frontiers instead of one operating point.

— 6 —

Conclusion and Perspectives

This project produced a reproducible framework for hybrid queries over tables and reviews, two baseline implementations, and two cascaded variants. The main lesson is architectural: relational pruning, batch prompting, and heterogeneous routing optimize different cost factors and are most effective together. The preliminary Trummer V1 result simultaneously improves macro F1 and reduces latency and calls.

The mathematical analysis also identifies a design debt. Trummer V1's categorical scores have confidence magnitudes only zero and two, making threshold calibration coarse. The next implementation should request or derive continuous per-candidate log-odds, calibrate them on held-out labelled data, and include uncertainty bounds. Other priorities are asynchronous batches, token-aware batch sizing, recall-safe retry policies, and a query optimizer that estimates candidate cardinality and chooses among row-wise, block-join, and cascaded plans.

The results are encouraging but preliminary. Confirmatory experiments with repeated runs, independent annotation, multiple domains, model families, and true resource measurements are necessary before making general performance claims.

— A —

Exact Prompt Templates

A.1 SUQL cheap row scorer

```
Act as a high-recall first-pass semantic filter. Decide whether this
    review
contains review-specific evidence for a Yes answer. Count direct wording
    ,
synonyms, described examples, and reasonable implications as evidence.
    Give
borderline but condition-specific evidence the benefit of the doubt. Do
    not
discard an explicit mention because it is negated, qualified, quoted, or
critical: this is evidence retrieval, not sentiment scoring. Do not
    count
genre/topic alone, generic praise or criticism, or mere lack of
    contradiction.
Return exactly one token: 1 for Yes, 0 for No.
Question: {semantic_question}
Review: {review[:1800]}
Answer:
```

A.2 Trummer cheap batch

```
Classify every explicit candidate pair below.
Predicate: {semantic_predicate}
Act as a high-recall first-pass semantic filter.
Answer YES for direct evidence, synonyms, described examples, or
    reasonable
implications. Answer UNCERTAIN when evidence is condition-specific but
    too weak
```

```
or ambiguous for a reliable YES. Answer NO only when review-specific
    support is
absent, generic, based only on genre/topic, or never discusses the
    predicate.
Return one line per candidate as CANDIDATE_ID: YES, NO, or UNCERTAIN.
Return every candidate exactly once and do not explain.
CANDIDATE_{id}:
Movie: {structured_movie_text}
Review: {review[:3500]}
Decisions:
```

A.3 Trummer expensive fallback

```
Evaluate the explicit candidate pairs below.
Predicate: {semantic_predicate}
Act as the final recall-first semantic filter. Answer YES for any
    concrete
review-specific support, including direct wording, synonyms, described
    examples,
and reasonable implications. Give borderline evidence the benefit of the
     doubt.
Answer NO when support is absent, generic, based only on genre/topic, or
     never
discusses the predicate itself. Lack of contradiction alone is not
    evidence.
Return one line per candidate as PAIR_ID: YES or PAIR_ID: NO.
Return every candidate exactly once. Do not add prose, JSON, or markdown
    .
PAIR_{id}:
Movie: {structured_movie_text}
Review: {review[:3500]}
Decisions:
```

— B —

Reproduction Commands

```
bash benchmarks/1q/run_local.sh --pull-models
bash benchmarks/run_local.sh --suite 5q --cheap-model gemma4:e2b \
  --expensive-model gemma4:26b
bash benchmarks/run_aker.sh --suite 10q --repetitions 10 \
  --methods "suql_baseline suql_v1 trummer_baseline trummer_v1"
```

Bibliography

[1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023.

[2] Zhoujun Cheng, Jungo Kasai, and Tao Yu. Batch prompting: Efficient inference with large language model apis. In *EMNLP Industry Track*, pages 792–810, 2023.

[3] Parker E. Glenn, Parag Pravin Dakle, Liang Wang, and Yoon Kim. BlendSQL: A scalable dialect for unifying hybrid question answering in relational algebra. In *Findings of ACL*, 2024.

[4] Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina J. Semnani, Chen Jie Yu, and Monica S. Lam. SUQL: Conversational search over structured and unstructured data with large language models. In *Findings of NAACL*, pages 4535–4555, 2024.

[5] Immanuel Trummer. Implementing semantic join operators efficiently, 2025.